Mp10 Web — Signature Pads

Capturing patient signatures from a Topaz pad

Structured Systems · www.structuredsystems.com

26 August 2026

Contents

1 About this guide

Mp10 Web can take a patient's signature directly from a **Topaz signature pad** plugged into the workstation, the same way the desktop application does.

This guide has two halves:

- **Using it** — for the person at the front desk. Two pages, no setup.
- **Installing it** — for whoever looks after the computers. This is marked **IT task** throughout.

There is one hard limitation to know before you start, because it decides who can use this at all:

> **Signature pads only work on Windows computers.** The software Topaz supplies is Windows-only. On a Mac, an iPad, or a phone, the **Sign** button will tell you so and point you at the other way of attaching a signature (see *If there is no pad*). This is a limitation of the pad's software, not of Mp10 Web, and there is no way around it.

2 Using it

2.1 Where the button is

Select an encounter, then choose **Sign**. The button is on both encounters screens:

- the **Encounters** list on the left-hand menu, and
- the **Encounters** list for one patient — both on the patient's record and in the Encounters window that opens from the patients list.

The button only lights up for an encounter that has no signature yet. If it is greyed out, hover over it: it will say *"This encounter already has a signature"*. That is deliberate — it stops one patient's signature being written over another's by accident.

To replace a signature that is already there, open the encounter itself and use the **Signature** panel on the encounter form.

2.2 Taking a signature

What the patient is shown before they sign — a consent text, and optionally a choice to make — is set up by your practice; see *Setting up the consent text* below. If nothing has been set up, the window goes straight to the signing area and steps 1 and 2 below simply do not appear.

1. **The consent text.** Select the encounter and choose **Sign**. The window shows the text your practice configured, one page at a time, with **Next** and **Back**. Let the patient read it. **Continue** stays greyed out until the last page has been reached — nobody can skip past the text.
2. **The choice**, if your practice configured one. The patient picks one of the options offered. It is one choice, not a tick-list: choosing a second option replaces the first. **Continue** stays greyed out until one is chosen.
3. **The signature.** The signing area appears; it says *"Connecting to the signature pad…"* for a moment. Ask the patient to sign **on the pad**. Their signature appears in the window as they write.
4. If they want to start again, choose **Clear** and let them sign again. This clears both the pad and the screen.
5. Choose **Save signature**.

The window confirms *"Signature saved"* and closes itself. The encounter's **Signature** column now shows a tick, and **Sign** greys out for that encounter.

If the patient changes their choice after signing, the signature is cleared and they are asked to sign again. This is deliberate, and the desktop application does the same: the signature has to belong to the choice actually made, or it is not evidence of anything. The window tells you when it happens.

What the patient chose is stored **with** the signature, along with every option they were offered — so a form printed later shows the decision they were actually making, not just which box ended up ticked.

2.3 Things it will not let you do

- **Saving an empty pad.** If nobody has signed, **Save signature** refuses and says so. A blank image is worse than none: the encounter would look signed when it is not.
- **Signing the wrong encounter.** The window's title shows the encounter number the signature will be attached to. Check it matches before saving.

2.4 If there is no pad

You can always attach a signature without a pad:

1. Open the encounter.
2. Use the **Signature** panel's **Capture / Upload** button.

3. On a computer this opens a file picker — choose a scanned image. On a phone or tablet it can open the camera, so you can photograph a signature on paper.

This is also the way to **replace** a signature that has already been taken.

2.5 When something is wrong

The signature window explains problems in place rather than failing silently. The message tells you which of these it is:

What it says	What it means	Who fixes it
Signature pads are only supported on Windows workstations	You are on a Mac, tablet or phone	Use the encounter form instead
Topaz's `SigWebTablet.js` was not found	The web server is missing a Topaz file	**IT task** — see below
SigWeb … is not running on this workstation	The Topaz software is not installed or not started on this PC	**IT task** — see below
Nothing has been signed yet	The pad is connected but nobody signed	Ask the patient to sign

In every one of these cases **nothing is saved** and the encounter stays unsigned, so it is always safe to close the window and try again.

3 Installing it — IT task

Two separate things have to be in place. They are independent, and the error messages above tell you which one is missing.

1. **On each Windows workstation** that has a pad: Topaz's **SigWeb**.
2. **On the web server**, once: Topaz's `SigWebTablet.js` file.

3.1 What SigWeb is

SigWeb is Topaz's browser-facing component. It installs as a Windows service that talks to the pad, and web pages drive it through a JavaScript file Topaz supplies. There is **no browser plug-in and no ActiveX**, so it works in Chrome, Edge and Firefox — but only on Windows.

3.2 1. Install SigWeb on the workstation

Download SigWeb from Topaz's site (https://www.topazsystems.com) and run the installer on each Windows PC that has a pad. Plug the pad in and confirm Windows recognises it.

> **Confirm with Topaz which component your pads need.** SigWeb is the usual answer for a browser-based application, but Topaz also ships SigPlusExtLite for some situations, and licensing terms differ. Check this against your model of pad and your licence before rolling it out to every workstation.

3.3 2. Put `SigWebTablet.js` on the web server

The SigWeb installation includes a JavaScript file called `SigWebTablet.js`. Copy it to the Mp10 Web server so the browser can load it:

```
<Mp10 Web>/frontend/public/sigweb/SigWebTablet.js
```

On a built/deployed site, the same file must end up in the served folder:

```
<Mp10 Web>/frontend/dist/sigweb/SigWebTablet.js
```

Anything in `public/` is copied into `dist/` by `npm run build`, so putting it in `public/` before a build covers both.

> **Why this file is not shipped with Mp10 Web.** It is Topaz's file, under Topaz's licence, and it changes between SigWeb versions. Taking it from the installer you actually ran keeps the browser and the workstation on the same version.

3.4 3. Check it

On a workstation with a pad, sign in to Mp10 Web, select an encounter with no signature and choose **Sign**.

- Signing area appears, no error → **working**.
- *"SigWebTablet.js was not found"* → step 2 was missed, or the file is in the wrong folder. Browse to `https://<your-server>/sigweb/SigWebTablet.js` — you should get JavaScript, not a "not found" page.
- *"SigWeb … is not running on this workstation"* → step 1 was missed on this PC, or the SigWeb service is stopped. Check Windows Services.

3.5 Setting up the consent text — IT task

Mp10 Web reads **exactly the same setting the desktop application reads**, so there is only one place to change it and both stay in step. It lives in the Mp10 System Registry (not the Windows registry):

Field	Value
Section	`TOPAZ`
Entry	`ENCOUNTER-SIGNATURE-DISPLAYMESSAGE`

Leave it unset and signatures are simply collected with no consent text.

3.5.1 Put the text in a file, not in the setting

> **The setting itself holds only 250 characters.** Anything longer is cut off without warning. A real consent will not fit, so the setting should hold the **full path to a file**, and the file holds the text. Mp10 reads the file every time, so editing the file is all that is needed to change the wording.

The file may be either plain text, or JSON if you want options as well.

Plain text — the whole file is the message:

```
I authorise the practice to provide the treatment discussed with me.
```

JSON — a message plus a choice for the patient to make:

```json
{
  "ShowTextOnPad": "CONSENT TO TREATMENT\n\nI authorise the practice to...",
  "Options": [
    { "id": 1, "Option": "I consent to the treatment described above" },
    { "id": 2, "Option": "I decline the treatment described above" }
  ],
  "NewPageForOptions": true,
  "NewPageForSignature": true
}
```

- `ShowTextOnPad` — the message. `\n` starts a new line. Length is not limited; it is paged automatically.
- `Options` — the choice. Each needs an `id` (what gets stored) and an `Option` (what the patient reads). Omit `Options` entirely for consent with no choice.
- `NewPageForOptions` / `NewPageForSignature` — whether each step starts a fresh page.

If the file is meant to be JSON but has a syntax error, nothing breaks: it is treated as plain text, so the patient sees the raw JSON instead of a formatted consent. If that is what you are seeing, the file has a typo in it.

3.5.2 Checking a change

Change the file, then take a signature on any encounter. The new wording appears immediately — there is nothing to restart and no cache to clear.

3.6 If a pad works in the desktop app but not here

The desktop application and Mp10 Web use **different** Topaz components. The desktop app talks to the pad directly; the browser needs SigWeb as well. A pad working in the desktop app therefore proves the hardware and the cable are fine — it does not mean SigWeb is installed. Do step 1 anyway.

3.7 What gets stored

The signature is stored as a **PNG image** on the encounter, in exactly the same place a scanned or photographed signature goes. Nothing about the rest of Mp10 changes, and the desktop application reads these signatures normally.